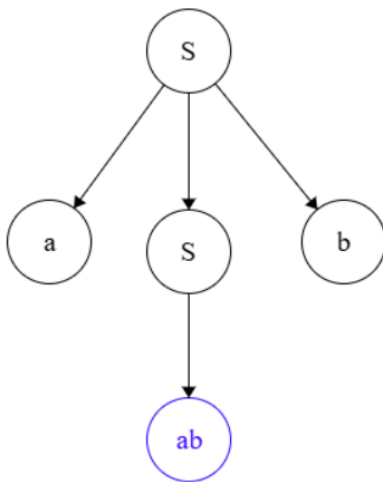


1. Consider the context-free grammar $G = (\{S\}, \{a, b\}, S, \{S \rightarrow aSb | bSa | SS | ab | ba\})$ Is it ambiguous? Justify your answer: if you think the answer is no, prove it. If you think the answer is yes, exhibit a string $w \in L(G)$ with two different parse trees for it.

- A CFG G is ambiguous iff $\exists w \in L(G)$ such that there are two distinct derivation trees for it.
- $S \rightarrow SS$ is the rule that shows ambiguity. Allows several possible partitions of a string into substrings, showing that the grammar provides different paths for splitting the string into substrings.
- This leads to the parsing of multiple trees for specific strings, which then cause ambiguity.

Proof Ex: (This is not providing the two different parse trees)

Take string xy , which can be parsed as a whole string for the other rules. It also can be split into x and y derived from separate occurrences of S



Exhibiting string $w = abab$, $w \in L(G)$

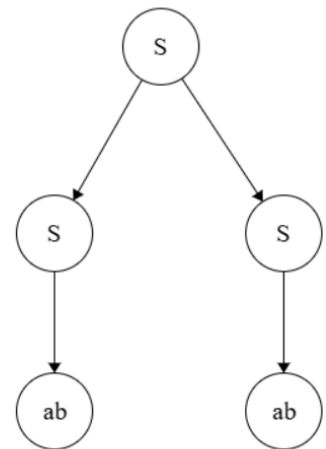
Parse tree on the left:

$S \rightarrow SS \rightarrow abab$

Parse tree on the right:

$S \rightarrow aSb \rightarrow a(ab)b$

These two parse trees are different for the string $abab$ which belongs to the language of the context free grammar. Therefore, this grammar is ambiguous as shown above.



2. Let $|x|_a$ be the number of occurrences of the symbol a in the string x .

1) Define a context-free grammar for the language $L = \{w \in \{0, 1\}^* : |w|_0 = |w|_1\}$

Strings comprising of 0's and 1's with exactly as many 0's as there are 1's.

Ex: 0011, ϵ , 10, 01, 1001

So I need a grammar that can balance the 0's and 1's while also having the empty string as well.

$S \rightarrow 1S0$ = For every 1 placed at the left of a substring, place a 0 at the right of the substring.

$S \rightarrow 0S1$ = For every 0 placed at the left of a substring, place a 1 at the right of the substring

$S \rightarrow SS$ = 2 concatenated strings that have balanced amounts of 0's and 1's are combined to then form a larger string that still accords to the rules

$S \rightarrow \epsilon$ = The empty string has no 0's or 1's, so it must also be a rule

So the resulting context free grammar will be $G = (\{S\}, \{0, 1\}, S, \{S \rightarrow 0S1 \mid 1S0 \mid SS \mid \epsilon\})$

2) Give a formal proof, or at least the key ideas of why your grammar generates L.

Need to prove that $L(G) \subseteq L$ and that every string that is generated by Grammar G is in L

Base Case:

$S \rightarrow \epsilon$ rule creates the empty string and is in L as there are no 0's and 1's.

Inductive Case: (balanced = equal amount of 0's and 1's)

- Assume that the strings u, v that are created by S are balanced with equal amounts of 0's and 1's in both u and v

- $S \rightarrow 0S1$: String with form 0u1 has additional 0 and additional 1. Therefore if u is balanced, 0u1 is also balanced.

- $S \rightarrow 1S0$: If u is balanced, then 1u0 is also balanced.

- $S \rightarrow SS$: Concatenation uv of two balanced strings will then remain balanced with same amount of 0's and 1's
Therefore all of the productions will have balanced amounts of 0's and 1's, which shows that all strings produced by G are in L.

Now need to prove the opposite, that $L \subseteq L(G)$

- If balanced string starts with 0 and ends with 1, you can write $w = 0u1$ (u is still assumed balanced)

- If w starts with 1 and ends with 0, you can write $w = 1u0$

- Otherwise, it is partitioned into substrings $w = xy$, which are smaller and created by the rule $S \rightarrow SS$

Through recursion with above rules, any string with a balanced amount of 0s and 1s can be derived using the productions $S \rightarrow 1S0$, $S \rightarrow 0S1$, and $S \rightarrow SS$. Therefore, all balanced strings in L are generated by the grammar

Since it has been proven $L \subseteq L(G)$ and $L(G) \subseteq L$, the grammar $G = \{\{S\}, \{0, 1\}, S, \{S \rightarrow 0S1 \mid 1S0 \mid SS \mid \epsilon\}\}$ therefore generates the language outlined in the problem.

3.

1) Define a context-free grammar for the language $L = \{a^n b^m c^{(n-m)} : n \geq m\}$

- Strings in L have zero or more a's, then zero or more b's, then number of c's = number of a's - number of b's

- Must always have as many a's as b's since there cannot be a negative amount of c's

- Strings in L: aaabbc, ab, ϵ , aabb, aaaaaabcccccc, ac

- Each a must be paired with a b or a c so the subtraction is remembered.

- So the two rules would be an a matched with a b or an a matched with a c. It cannot start with a b or c as it would be empty or invalid. Also the empty string would be the base case and a rule in itself too.

Rules:

$S \rightarrow aSb$ bc match one a to one b

$S \rightarrow aSc$ bc match one a to one c

$S \rightarrow \epsilon$ bc it is the base case $\rightarrow$ termination

The context free grammar would be $G = \{\{S\}, \{a,b,c\}, S, \{S \rightarrow aSc \mid aSb \mid \epsilon\}\}$

2) Find a CFG for the language $L = \{xyzy^R x^R : x \in \{a, b\}^*, y \in \{c, d\}^*, z \in \{a\}^*\}$

Substring $x \in \{a, b\}^*$ is string of a's and b's

Substring $y \in \{c, d\}^*$ is string of c's and d's

Substring $z \in \{a\}^*$ is string of a's

The string is symmetric with the center being the z substring, as:

$$x = (ab)^* \quad y = (cd)^* \quad z = a^* \quad y^R = (dc)^* \quad x^R = (ba)^*$$

- Need four nonterminals for the grammar, $\{S, X, Y, Z\}$ where S is start symbol, X creates the x and reversed x^R portions, Y creates the y and y^R portions, and Z creates the central part of the string consists of a's.
- Note that Z could be empty with 0 a's and therefore has an empty string in its rules, but nothing else can have it, as it simply moves on to a new rule to maintain symmetry
- Terminals = $\{a, b, c, d\}$

Production Rules:

- $S \rightarrow aSa \mid bSb \mid Y$ Constructs symmetric $\{a, b\}$ strings around the remaining string Y
 $Y \rightarrow cYc \mid dYd \mid Z$ Constructs symmetric $\{c, d\}$ strings around the remaining string Z
 $Z \rightarrow aZ \mid \epsilon$ Constructs the central sequence of a's, which can also be empty.

Therefore the grammar above is represented as $G = (\{S, X, Y, Z\}, \{a, b, c, d\}, S, P)$ with P being equal to $\{S \rightarrow aSa \mid bSb \mid Y, Y \rightarrow cYc \mid dYd \mid Z, Z \rightarrow aZ \mid \epsilon\}$

4. Define a NPDA for the language $L = \{w \in \{a, b\}^* : |w|_a = 3|w|_b\}$

- Language where strings have number of a's and 3 times the number of b's
- To balance the amount of a's and bs accordingly, use a stack. For each a, push one 'A' to the stack and for each b, pop three 'A's from the stack.
- If the stack is empty at the end of the string, then it is accepted.

NPDA $M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$ where

$Q = \{q_0, q_1, q_2, q_{\text{accept}}\}$ (states)

$\Sigma = \{a, b\}$ (symbols)

$\Gamma = \{A, Z_0\}$ (stack symbols)

q_0 = initial state

Z_0 = Initial stack symbol on the bottom of the stack.

$F = \{q_{\text{accept}}\}$ (accepting state)

δ (Transitions):

Reading A:

$\delta(q_0, a, Z) = \{\{q_0, Az\}\}$ for $z \in \Gamma$

- Push an A for each a

Reading B:

$$\delta(q_0, b, A) = \{(q_1, \varepsilon)\}$$

$$\delta(q_1, \varepsilon, A) = \{(q_2, \varepsilon)\}$$

$$\delta(q_2, \varepsilon, A) = \{(q_0, \varepsilon)\}$$

- For reading one b, need to pop 3 A's. Use empty string transitions after the first pop to pop the other 2 A's and navigate through the states q_1 and q_2 .

$$\delta(q_0, \varepsilon, Z_0) = \{(q_{\text{accept}}, Z_0)\}$$

- After the string has been inputted, accept the string only if the stack has the bottom indicator Z_0 .

$$q_{\text{accept}} \in F$$

States: $Q = \{q_0, q_1, q_2, q_{\text{accept}}\}$

Alphabet: $\Sigma = \{a, b\}$

Stack Alphabet: $\Gamma = \{A, Z_0\}$

Transitions:

$$\delta(q_0, a, Z) = \{(q_0, Az)\} \text{ for } z \in \{A, Z_0\}$$

$$\delta(q_0, b, A) = \{(q_1, \varepsilon)\}$$

$$\delta(q_1, \varepsilon, A) = \{(q_2, \varepsilon)\}$$

$$\delta(q_2, \varepsilon, A) = \{(q_0, \varepsilon)\}$$

$$\delta(q_0, \varepsilon, Z_0) = \{(q_{\text{accept}}, Z_0)\}$$

Start State: q_0

Bottom of Stack Indicator: Z_0

Accepting State: q_{accept}

The NPDA described above accepts the language L stated in the problem.

5. Define a NPDA for the language $L = \{uv \in \{0,1\}^* : |u| = |v| \wedge u \neq v^R\}$

- Even-length strings w that are split into two equal substrings u and v

- The first half substring u is not equal to the reverse of the second half substring v^R

- Stack data structure is used to remember the first half substring u , then when iterating through the second half substring and compare with symbols in the stack.

NPDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ where:

$$Q = \{q_0, q_1, q_{\text{accept}}\}$$

- q_0 will push the first half, q_1 will pop from the stack and compare with the second half, and q_{accept} will accept the string if all rules are met.

$$\Sigma = \{0, 1\} \text{ (Input } \Sigma)$$

$$\Gamma = \{0, 1, Z_0\} \text{ (Stack } \Sigma)$$

Q_0 = Start state

Z_0 = Stack start symbol

$F = \{q_{\text{accept}}\}$ (Accepting states)

δ (Transition Function):

$\delta(q_0, a, x) = \{(q_0, ax)\}$ for $x \in \Gamma$

- For each of the input symbols $a = 0$ or 1 , push onto the stack

$\delta(q_0, \epsilon, x) = \{(q_1, x)\}$

I am not sure how the middle of the string will be calculated. I don't understand how the string is found to be split in half. I am going to submit anyways.

For each symbol $a \in \{0,1\}$ compare with the top of the stack

If it matches up, pop it and keep going

$\delta(q_1, a, a) = \{(q_1, \epsilon)\}$

If there is a mismatch then accept it

$\delta(q_1, a, b) = \{(q_{\text{accept}}, b)\}$ for $a \neq b$

- Accept the string after a mismatch is found, and go to state q_{accept}

$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

States: $Q = \{q_0, q_1, q_{\text{accept}}\}$

Input Alphabet: $\Sigma = \{0,1\}$

Stack Alphabet: $\Gamma = \{0,1,Z_0\}$

Start State: q_0

Start Stack Symbol: Z_0

Accepting state: $F = \{q_{\text{accept}}\}$

Transitions:

$\delta(q_0, a, x) = \{(q_0, ax)\}$ for $x \in \Gamma$

$\delta(q_0, \epsilon, x) = \{(q_1, x)\}$

$\delta(q_1, a, a) = \{(q_1, \epsilon)\}$

$\delta(q_1, a, b) = \{(q_{\text{accept}}, b)\}$ for $a \neq b$

The NPDA described above accepts the language L stated in the problem.

$\subseteq \wedge \in \forall \exists \neg \Sigma \epsilon \cup$